LEVEL 1 — BASIC

Task 1: Basic Calculator

Code:-

```
import java.util.Scanner;

class Calculator {
    public double add(double a,
double b) {         return a + b;     }

    public double subtract(double a,
double b) {         return a - b;     }

    public double multiply(double a,
double b) {         return a * b;     }

    public double divide(double a,
double b) {         if (b == 0) {
            throw new ArithmeticException("Cannot divide
by zero");
        }
        return a / b;
    }
}

public class Main {
    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
        Calculator calc = new Calculator();

        System.out.print("Enter first number: ");
double a = sc.nextDouble();

        System.out.print("Enter second number:
");         double b = sc.nextDouble();

        System.out.print("Choose operation (+, -, *, /):
");         char op = sc.next().charAt(0);

        try {
            double result = switch (op) {
case '+' -> calc.add(a, b);
case '-' -> calc.subtract(a, b);
case '*' -> calc.multiply(a, b);
case '/' -> calc.divide(a, b);
                default -> throw new
IllegalArgumentException(
                    "Invalid operation");
            };
```

```
            System.out.println("Result: " + result);
        } catch (ArithmeticException |
IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }

        sc.close();
    }
}
```

Sample Output

```
Enter first number: 20
Enter second number: 5
Choose operation (+, -, *, /): +
Result: 25
```

Task 2: Simple Number Guessing Game

Code

```
import java.util.Random;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Random random = new Random();
        Scanner sc = new Scanner(System.in);

        int number =
random.nextInt(100) + 1;        int
attempts = 7;        boolean guessed
= false;

        System.out.println("Guess a number between 1
and 100.");
        System.out.println("You have " + attempts + "
attempts.");

        for (int i = 1; i <= attempts; i++) {
System.out.print("Attempt " + i + ": ");

            if (!sc.hasNextInt()) {
                System.out.println("Please enter a valid
integer.");                sc.next();                i--;
continue;            }            int guess = sc.nextInt();

            if (guess < 1 || guess > 100) {
                System.out.println("Enter a number from 1 to
100.");                i--;                continue;            }

            if (guess == number) {
```

```
            System.out.println("Correct! You
won.");                guessed = true;
break;
        } else if (guess < number) {
            System.out.println("Too low!");
        } else {
            System.out.println("Too high!");
        }
    }

    if (!guessed) {
        System.out.println("Game over! Number was "
+ number);
    }

    sc.close();
  }
}
```

Output

```
Guess a number between 1 and 100.
You have 7 attempts.
Attempt 1: 40
Too low!
Attempt 2: 70
Too high!
Attempt 3: 55
Correct! You won.
```

Task 3: Factorial Calculation Using Recursion

Code

```
import java.util.Scanner;

public class Main {
  public static long factorial(int n) {
if (n < 0 || n > 20) {
        throw new IllegalArgumentException(
            "Enter a number between 0 and 20");
    }

    if (n == 0 || n == 1) {
return 1;       }

    return n * factorial(n - 1);
}

  public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter a number:
");         int n = sc.nextInt();

        try {
            System.out.println("Factorial: " + factorial(n));
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }

        sc.close();
    }
}
```

Sample Output

```
Enter a number: 5
Factorial: 120
```

LEVEL 2 — INTERMEDIATE

Task 1: Employee Management System (CRUD)

Code

```
import java.util.ArrayList;
import java.util.Scanner;

class Employee {
private int id;
private String name;
private double
salary;

    public Employee(int id, String name, double
salary) {         this.id = id;         this.name =
name;         this.salary = salary;     }

    public int getId()
{         return id;     }

    public void update(String name, double
salary) {         this.name = name;
this.salary = salary;     }

    @Override
    public String toString() {
        return "ID: " + id + ", Name: " + name
                + ", Salary: " + salary;
    }
}

public class Main {
```

```
    static ArrayList<Employee> employees = new
ArrayList<>();     static Scanner sc = new
Scanner(System.in);

    static Employee find(int id) {
for (Employee e : employees) {
if (e.getId() == id) {
return e;
            }
        }
        return null;
}

    static void add() {
        System.out.print("Enter ID:
");         int id = sc.nextInt();
sc.nextLine();

        if (find(id) != null) {
            System.out.println("Employee ID already
exists.");             return;         }

        System.out.print("Enter name: ");
        String name = sc.nextLine();

        System.out.print("Enter salary: ");
double salary = sc.nextDouble();

        if (salary < 0) {
            System.out.println("Salary cannot be
negative.");             return;         }
employees.add(new Employee(id, name, salary));
System.out.println("Employee added.");
    }

    static void view() {         if
(employees.isEmpty()) {
            System.out.println("No employee records
found.");             return;
        }
        for (Employee e : employees) {
System.out.println(e);
        }
    }

    static void update() {
        System.out.print("Enter employee
ID: ");         int id = sc.nextInt();
sc.nextLine();
```

```
        Employee e =
find(id);         if (e == null)
{
            System.out.println("Employee not
found.");             return;         }

        System.out.print("Enter new name: ");
        String name = sc.nextLine();

        System.out.print("Enter new salary: ");
double salary = sc.nextDouble();

        if (salary < 0) {
            System.out.println("Salary cannot be
negative.");             return;         }

        e.update(name, salary);
        System.out.println("Employee updated.");
    }

    static void delete() {
        System.out.print("Enter employee
ID: ");         int id = sc.nextInt();

        Employee e =
find(id);         if (e != null) {
employees.remove(e);
            System.out.println("Employee deleted.");
        } else {
            System.out.println("Employee not found.");
        }
    }

    public static void main(String[] args) {
while (true) {
            System.out.println(
                "\\n1.Add  2.View  3.Update  4.Delete
5.Exit");
            System.out.print("Choose:
");             int choice = sc.nextInt();

            switch (choice) {
case 1 -> add();
case 2 -> view();
case 3 -> update();
case 4 -> delete();
case 5 -> {
sc.close();
return;
                }
```

```
                default -> System.out.println("Invalid
choice.");
            }
        }
    }
}
```

Output

```
1.Add  2.View  3.Update  4.Delete  5.Exit
Choose: 1
Enter ID: 101
Enter name: Aniket
Enter salary: 25000
Employee added.

1.Add  2.View  3.Update  4.Delete  5.Exit
Choose: 2
ID: 101, Name: Aniket, Salary: 25000.0
```

Task 2: File Handling — Reading and Writing

Code

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class Main {
    public static void main(String[] args)
{        Path input = Path.of("input.txt");
        Path output = Path.of("output.txt");

        try {            String content =
Files.readString(input);

            long lines = content.isEmpty()
                    ? 0
                    : content.lines().count();

            String trimmed = content.trim();
long words = trimmed.isEmpty()
                    ? 0
                    : trimmed.split("\\\\s+").length;

            String result = "Lines: " + lines
                    + "\\nWords: " + words;

            Files.writeString(output, result);

            System.out.println("Processing completed.");
```

```
            System.out.println(result);
        } catch (IOException e) {
            System.out.println("File error: " +
e.getMessage());
        }
    }
}
```

Output

Assume input.txt contains:
Java is easy
I am learning Java

Console output:
Processing completed.
Lines: 2
Words: 7

output.txt:
Lines: 2
Words: 7

Task 3: Simple Banking Application

Code

```
import java.util.Scanner;

class BankAccount {
private double
balance;

    public BankAccount(double
initialBalance) {        if (initialBalance <
0) {
            throw new IllegalArgumentException(
                "Initial balance cannot be negative");
        }
        balance =
initialBalance;     }

    public void deposit(double
amount) {        if (amount <= 0) {
            System.out.println("Amount must be
positive.");            return;
        }
        balance += amount;
        System.out.println("Deposit successful.");
```

```
    }

    public void withdraw(double
amount) {         if (amount <= 0) {
            System.out.println("Amount must be positive.");
        } else if (amount > balance) {
            System.out.println("Insufficient balance.");
        } else {
            balance -= amount;
            System.out.println("Withdrawal successful.");
        }
    }

    public void checkBalance() {
        System.out.printf("Balance: ₹%.2f%n", balance);
    }
}

public class Main {
    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
        BankAccount account = new BankAccount(1000);

        while (true) {
            System.out.println(
                "\\n1.Deposit 2.Withdraw 3.Balance 4.Exit");
            System.out.print("Choose:
");              int choice = sc.nextInt();

            switch (choice) {
case 1 -> {
                    System.out.print("Amount: ");
account.deposit(sc.nextDouble());
                }
                case 2 -> {
                    System.out.print("Amount: ");
account.withdraw(sc.nextDouble());
                }
                case 3 ->
account.checkBalance();
case 4 -> {                     sc.close();
                    return;
                }
                default -> System.out.println("Invalid
choice.");
            }
        }
    }
}
```

Output

```
1.Deposit 2.Withdraw 3.Balance 4.Exit
Choose: 1
Amount: 500
Deposit
successful.

1.Deposit 2.Withdraw 3.Balance 4.Exit
Choose: 3
Balance: ₹1500.00

1.Deposit 2.Withdraw 3.Balance 4.Exit
Choose: 2
Amount: 2000
Insufficient balance.
```

LEVEL 3 —
ADVANCED

Task 1: Library Management System with JDBC

Code

```
import java.sql.*;

public class LibraryDAO {
    private static final String URL =
            "jdbc:mysql://localhost:3306/library_db";
private static final String USER = "root";
private static final String PASSWORD =
"YOUR_PASSWORD";

    private Connection connect() throws SQLException {
        return DriverManager.getConnection(URL,
USER, PASSWORD);     }

    public void addBook(String title, String
author)            throws SQLException {
        String sql =
                "INSERT INTO Books(title, author) VALUES
(?, ?)";

        try (Connection con = connect();
             PreparedStatement ps =
con.prepareStatement(sql)) {            ps.setString(1,
title);            ps.setString(2, author);
ps.executeUpdate();
        }
    }

    public void registerUser(String name) throws
SQLException {        String sql = "INSERT INTO
Users(name) VALUES (?)";

        try (Connection con = connect();
             PreparedStatement ps =
con.prepareStatement(sql)) {            ps.setString(1,
name);            ps.executeUpdate();
        }
    }

    public void viewBooks() throws
SQLException {        String sql = "SELECT *
FROM Books";

        try (Connection con = connect();
             PreparedStatement ps =
con.prepareStatement(sql);
```

```
            ResultSet rs = ps.executeQuery())
{             while (rs.next()) {
                System.out.println(
rs.getInt("id") + " | " +
rs.getString("title") + " | " +
rs.getString("author") + " | Available: " +
rs.getBoolean("available"));
            }
        }
    }

    public void borrowBook(int bookId, int
userId)             throws SQLException {
        String update =
                "UPDATE Books SET available = FALSE " +
                "WHERE id = ? AND available = TRUE";

        try (Connection con =
connect()) {
con.setAutoCommit(false);
try {
                try (PreparedStatement ps =
con.prepareStatement(update)) {
ps.setInt(1, bookId);                     if
(ps.executeUpdate() != 1) {
throw new SQLException(
                            "Book not found or unavailable.");
                    }
                }

                String insert =
                    "INSERT INTO Transactions(book_id,
user_id) " +
                    "SELECT ?, id FROM Users WHERE id =
?";

                try (PreparedStatement ps =
con.prepareStatement(insert)) {
ps.setInt(1, bookId);                     ps.setInt(2,
userId);                     if (ps.executeUpdate()
!= 1) {
                        throw new SQLException("User not
found.");
                    }
                }
con.commit();
            } catch (SQLException
e) {                 con.rollback();
throw e;                } finally {
                con.setAutoCommit(true);
```

```
        }
      }
    }

    public void returnBook(int bookId, int
userId)            throws SQLException {
        String sql =
                "UPDATE Transactions SET returned_at =
  CURRENT_TIMESTAMP " +            "WHERE book_id = ?
          AND user_id = ? AND returned_at IS NULL";

        try (Connection con =
connect()) {
con.setAutoCommit(false);
try {
            try (PreparedStatement ps =
con.prepareStatement(sql)) {
ps.setInt(1, bookId);
ps.setInt(2, userId);                    if
(ps.executeUpdate() != 1) {
throw new SQLException(
                        "No active borrowing record found.");
                }
            }

            try (PreparedStatement ps =
con.prepareStatement(
                    "UPDATE Books SET available = TRUE
WHERE id = ?")) {                    ps.setInt(1, bookId);
ps.executeUpdate();
            }
con.commit();
        } catch (SQLException
e) {              con.rollback();
throw e;            } finally {
            con.setAutoCommit(true);
        }
      }
    }
}
```

Output

```
Example usage: LibraryDAO library
= new LibraryDAO();
library.addBook("Java Basics",
"James");
library.registerUser("Aniket");
library.viewBooks();
```

Sample listing:
1 | Java Basics | James | Available: true

After borrowing book ID 1 for user ID 1:
1 | Java Basics | James | Available: false

After returning the book:

1 | Java Basics | James | Available: true

Task 2: Multithreaded Chat Application

Code

```
import java.io.*;
import java.net.*;
import
java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public class ChatServer {
    private static final int PORT = 5000;
    private static final Set<PrintWriter> clients =
ConcurrentHashMap.newKeySet();

    public static void main(String[] args) throws
IOException {         try (ServerSocket server = new
ServerSocket(PORT)) {
            System.out.println("Chat server started on port " +
PORT);              while (true) {
                Socket socket = server.accept();
                Thread clientThread =
                        new Thread(() -> handleClient(socket));
clientThread.start();
            }
        }
    }

    private static void handleClient(Socket socket) {
        PrintWriter out = null;
try (socket;
             BufferedReader in = new BufferedReader(
                 new
InputStreamReader(socket.getInputStream()))) {
out = new PrintWriter(socket.getOutputStream(), true);
clients.add(out);              broadcast("A user joined the
chat.");

            String message;
            while ((message = in.readLine()) != null) {
broadcast(message);
```

```
            }
        } catch (IOException e) {
            System.out.println("Client disconnected.");
        } finally {            if (out
!= null) {
clients.remove(out);
            }
            broadcast("A user left the chat.");
        }
    }

    private static void broadcast(String
message) {        for (PrintWriter client :
clients) {            client.println(message);
        }
    }
}
```

Output

```
Server console:
Chat server started on port 5000
Client 1:
Connected. Type messages; /quit to exit.
Hello everyone!

Client 2:
A user joined the chat.
Hello everyone!
```

Task 3: Binary Search Tree (BST) Implementation

Code

```
public class Main {
static class TreeNode {
int data;        TreeNode
left, right;

        TreeNode(int data) {
this.data = data;
        }
    }

    static class BST {
private TreeNode root;

        public void insert(int value) {
root = insertNode(root, value);
}
```

```
    private TreeNode insertNode(TreeNode node,
int value) {            if (node == null) {
            return new TreeNode(value);
        }
        if (value < node.data) {
            node.left = insertNode(node.left, value);
        } else if (value > node.data) {
            node.right = insertNode(node.right, value);
        }
        return node;
}

    public boolean search(int value) {
        TreeNode current = root;
while (current != null) {
            if (current.data == value) return true;
current = value < current.data
                    ? current.left : current.right;
        }
        return false;
}

    public void delete(int value) {
root = deleteNode(root, value);
}

    private TreeNode deleteNode(TreeNode node,
int value) {            if (node == null) return null;

        if (value < node.data) {
            node.left = deleteNode(node.left, value);
        } else if (value > node.data) {
            node.right = deleteNode(node.right, value);
        } else {
            if (node.left == null) return node.right;
            if (node.right == null) return node.left;

            node.data = minValue(node.right);
node.right = deleteNode(node.right, node.data);
        }
return node;        }

    private int minValue(TreeNode node) {
while (node.left != null) node = node.left;
return node.data;        }

    public void inOrder() {
inOrderTraversal(root);
System.out.println();
    }
```

```
    private void inOrderTraversal(TreeNode
node) {         if (node != null) {
inOrderTraversal(node.left);
            System.out.print(node.data + " ");
inOrderTraversal(node.right);
        }
    }

    public void preOrder() {
preOrderTraversal(root);
System.out.println();
    }

    private void preOrderTraversal(TreeNode
node) {         if (node != null) {
            System.out.print(node.data + " ");
preOrderTraversal(node.left);
preOrderTraversal(node.right);
        }
    }

    public void postOrder() {
postOrderTraversal(root);
System.out.println();
    }

    private void postOrderTraversal(TreeNode
node) {         if (node != null) {
postOrderTraversal(node.left);
postOrderTraversal(node.right);
System.out.print(node.data + " ");
        }
    }
  }

  public static void main(String[] args) {
    BST tree = new BST();
tree.insert(50);
tree.insert(30);
tree.insert(70);
tree.insert(20);
tree.insert(40);
tree.insert(60);
tree.insert(80);

    System.out.print("In-order: ");
tree.inOrder();
    System.out.print("Pre-order: ");
tree.preOrder();
    System.out.print("Post-order: ");
tree.postOrder();
```

```
        System.out.println("Search 40: " +
tree.search(40));          tree.delete(30);
        System.out.print("After deleting 30: ");
tree.inOrder();
    }
}
```

Output

```
In-order: 20 30 40 50 60 70 80
Pre-order: 50 30 20 40 70 60 80
Post-order: 20 40 30 60 80 70 50
Search 40: true
After deleting 30: 20 40 50 60 70 80
```